

Cómo construimos el prototipo de reservas con ayuda de IA

Sesión de diseño asistido · Turismo Social

HTML · CSS · JS

Figma MCP

Claude

Contenido de esta presentación

01	El encargo	Qué queríamos conseguir y bajo qué restricciones
02	El proceso	Cómo iteramos de v1 a v7l en una única sesión
03	Prompts que funcionaron	Las instrucciones que produjeron los mejores resultados
04	Criterios de calidad	Cómo evaluamos si cada resultado era bueno
05	El artefacto final	Especificación completa del prototipo
06	Patrones reutilizables	Lo que podemos aplicar en futuros proyectos

01 El encargo

Qué se quería diseñar

Un prototipo HTML interactivo del flujo completo de reserva de viajes para la aplicación de **Turismo Social** — entidad pública que gestiona viajes subvencionados para mayores y personas con discapacidad.

Requisito	Detalle
Plataforma	HTML single-file, sin dependencias externas
Dispositivo	375 × 736 px, fondo gris exterior, scroll interno
Fidelidad	Visual y funcional al sistema de diseño Figma existente
Pasajeros	Múltiples, con lógica de emparejamiento de habitaciones
Pago	4 métodos: tarjeta, Bizum, transferencia, agencia/cajero
Assets	Imágenes y SVGs embebidos como base64

Flujo de 7 pasos

01	Identificación	Añadir pasajeros con DNI + clave
02	Búsqueda	Filtros: transporte, mascota, zona, fechas
03	Resultados	Cards de hotel con tags de disponibilidad
04	Configuración	Habitaciones + servicios especiales por pasajero
05	Datos personales	Tel., email y condiciones por pasajero
06	Pago	Bottom sheet → pasarela simulada → éxito
07	Confirmación	Localizador, estado, descarga de documentación

02 El proceso · Iteraciones

v1–v2	Base estructural Arquitectura de navegación con history[] propio. Decisión: display:none/flex en lugar de routing para mantener todo en un fichero.
v3	Assets reales del Figma Captura de screenshots + get_design_context para extraer tokens CSS exactos. SVGs leídos del upload, imágenes embebidas como base64.
v4	Hotel cards y tags Tags como SVG escalado → fallaban. SVG reescalado → seguía fallando. Solución: span HTML puro con estilos inline exactos del Figma.
v5–v6	Paso 3 · Habitaciones Tres iteraciones: lista con exclusiones → sin exclusiones → "queda sin asignar". Auto-asignación al entrar según nº de pasajeros.
v7a–v7e	Tags y validaciones Correcciones de tamaño, color y wrapping de tags. Validaciones en paso 2 sin asteriscos previos. Date-range picker custom.
v7f–v7i	Modales y paso 6 Modales de confirmación (volver/salir) centradas fieles al Figma. Bug de resumen duplicado en paso 6 resuelto con bottom sheet sobre s6.
v7j–v7l	Pasarela de pago Flujo completo: bottom sheet → selector → formulario → éxito → vuelta a la app. Header oculto en pantallas browser simuladas.

■ Patrón de iteración óptimo

Los cambios pequeños y verificados generaron menos bugs que los cambios grandes. El ciclo más eficiente fue: posicionarse en Figma → capturar → implementar → probar → describir síntoma → fix quirúrgico.

03 Prompts que funcionaron

Posicionarse en Figma

```
""Me posiciono en el frame X en Figma, cógelo directamente de ahí""
```

→ Producía tokens exactos sin interpretación libre. Mucho mejor que describir el componente en texto.

Comportamiento causa-efecto

```
""cuando un pasajero cambia su compañero y elige otro que ya tiene asignado, se actualiza quitando al anterior""
```

→ Describir X → Y fue más claro que describir el estado final.

Síntoma, no causa

```
""el prototipo no navega hacia el paso 6, se queda colgado""
```

→ Describir qué ocurre (síntoma) permitió un mejor diagnóstico que intentar explicar qué está mal.

Anclar al producto real

```
""necesito que sean como las del diseño, porque en la app ya están desarrolladas así""
```

→ Anclar a la implementación real redujo la interpretación libre y dio autoridad a las especificaciones.

Casos exhaustivos

```
""Si 1 pasajero → doble + aleatorio. Si par → emparejados. Si impar → pares + último aleatorio""
```

→ Especificar todas las condiciones como lista if/else fue mucho más preciso que describir el comportamiento general.

Preguntar antes de implementar

```
""si tienes alguna duda coméntamela antes de continuar""
```

→ Evitó implementaciones incorrectas en flujos complejos como el paso 3 y la pasarela de pago.

04 Criterios de calidad

Fidelidad visual

- Colores: tokens exactos del Figma, nunca aproximaciones (#4D6E05 no #4a7000)
- Tipografía: tamaños de fuente coincidentes frame a frame
- Componentes: mismo comportamiento visual que la app real

Sin errores de ejecución

- Cero errores de JS en consola
- Navegación completamente lineal, sin bucles ni pantallas en blanco
- Datos persistentes al navegar atrás

Realismo de simulación

- Dispositivo con tamaño fijo, scroll interno, header sticky
- Pasarelas de pago que ocultan el chrome de la app
- Validaciones idénticas a las de la app real

Lógica de negocio correcta

- Reglas de emparejamiento de habitaciones exactas
- Precios calculados correctamente (+15€ individual)
- Estados de pago (Realizado/Pendiente) según método elegido

05 El artefacto final · Tokens de diseño

El prototipo es un único fichero HTML (~1.3 MB) que funciona en cualquier navegador sin servidor ni dependencias externas.

Paleta de colores

Token	Valor	Uso
--g	#03594C	Verde principal · botones, títulos, iconos
--gl	#D6E2DF	Verde claro · fondos de cards, chips
--gm	#88A9A1	Verde medio · bordes, texto secundario
--bg	#FAFAFA	Fondo general de la app
--sf	#F7F7F7	Surface · cards y formularios
--yw	#F1EDB1	Amarillo · fondos de totales, botones activos
--tx	#3B3E43	Texto principal
--mu	#656565	Texto muted / labels
--re	#C62828	Errores y alertas

Dispositivo simulado

Ancho	375 px	Alto	736 px (Android estándar)
Border-radius	24 px	Tipografía	Inter 400/500/600/700

05 El artefacto final · Lógica de habitaciones

Auto-asignación al entrar en el paso 3

Condición	Resultado
1 pasajero	Habitación doble + compañero aleatorio de otra reserva
Número par de pasajeros	Emparejados entre sí: 0↔1, 2↔3, etc.
Número impar de pasajeros	Pares emparejados + último recibe aleatorio

Reglas de cambio de compañero

A elige a B como compañero	B queda emparejado con A. El anterior compañero de A queda "Sin asignar" (chip rojo).
A cambia a Individual	Su compañero anterior queda "Sin asignar", no se le asigna aleatorio automáticamente.
A vuelve a Doble (1 pax)	Se recupera automáticamente el compañero aleatorio.
A vuelve a Doble (+1 pax)	Queda sin asignar hasta que el usuario elija manualmente.
Continuar con Sin asignar	Error global al fondo: "Los pasajeros no están correctamente distribuidos."

05 El artefacto final · Flujo de pago

El botón "Continuar al pago" del resumen abre un bottom sheet sobre la misma pantalla (sin navegar a una nueva), evitando la duplicación de contenido.

Desde	Acción	Resultado
Resumen (paso 5)	"Continuar al pago"	Abre bottom sheet sobre la pantalla actual
Bottom sheet	Pago online	Navega a pantalla browser · selector de método
Selector (browser)	Tarjeta	→ Formulario tarjeta → Pagar ahora → Éxito
Selector (browser)	Bizum	→ Campo teléfono → Continuar → Éxito
Selector (browser)	Transferencia	→ Selector banco → Continuar → Éxito
Bottom sheet	Banco/caja	Paso 7 directo · estado Pendiente
Bottom sheet	Agencia de viajes	Paso 7 directo · estado Pendiente
Pantalla éxito	"Volver a la app"	Paso 7 · estado Realizado · header restaurado

Truco · Header en pasarela externa

Las pantallas de browser simulado (s6b–s6f) ocultan el header de la app mediante `showScr()`. Al volver al paso 7, el header se restaura automáticamente. Esto simula que el usuario ha "salido" a una web externa.

06 Patrones generalizables



01

Figma como fuente de verdad

Posicionarse sobre el elemento en Figma y capturar screenshot + design_context produce resultados más precisos que cualquier descripción textual. Especialmente útil para colores exactos, espaciados y comportamiento de componentes.



02

HTML single-file con base64

Para prototipos que deben funcionar sin servidor: convertir todas las imágenes a base64 y embeberlas. Fichero grande (~1-2MB) pero completamente autónomo. Perfecto para sesiones de test con usuarios.



03

Historial de navegación propio

Implementar history[] propio en lugar de usar el historial del navegador permite modales de confirmación al volver atrás y mantiene el estado de los datos entre pantallas.



04

Tags como spans, nunca como SVGs

Los SVGs con texto del Figma no escalan bien en HTML. Siempre convertir tags/badges/pills a span CSS inline: background, color, height:28px, width:fit-content, border-radius:99px.



05

Bottom sheet sobre pantalla actual

Para no duplicar contenido entre pantallas, los sheets deben abrirse sobre la pantalla actual en lugar de navegar a una nueva. Esto evita el bug de "pantalla duplicada" que ocurrió en el paso 6.



06

Pantallas browser-simuladas

Para pasos externos (pasarelas de pago, 2FA), ocultar el chrome de la app al entrar en esas pantallas y restaurarlo al volver. Mejora significativamente el realismo del prototipo.

06 Cómo repetirlo · Checklist de arranque

Lo que se necesita para replicar este workflow en otro proyecto similar:

- **Fichero Figma activo** Con el MCP de Figma conectado en Claude. Permite capturar screenshots y tokens exactos.
- **SVGs exportados** Iconos y tags exportados de Figma como ficheros .svg subidos en la conversación.
- **Imágenes del producto** JPG/PNG de los assets reales (fotos de hotel, etc.). Se convierten a base64 automáticamente.
- **Capturas de sistemas externos** Pantallazos de pasarelas de pago, formularios externos, etc., para diseñar las simulaciones.
- **Lógica de negocio en formato condicional** Describir las reglas como lista exhaustiva de casos (si X → Y) antes de implementar.
- **Preguntar antes de flujos complejos** Pedir que Claude pregunte dudas antes de implementar cualquier flujo con múltiples condiciones.
- **Fichero de referencia de versión anterior** Si es una iteración, tener el HTML anterior disponible para aplicar parches quirúrgicos.

Resultado de esta sesión

~1.3 MB HTML autónomo sin dependencias	7 pasos Flujo completo de reserva	v7l Versión final tras 12 iteraciones	4 métodos de pago simulados con confirmación
--	---	---	--